

Proyecto PropTech - "Insignia Inmo"

Modelado Completo del Proyecto

1. Stack Tecnológico

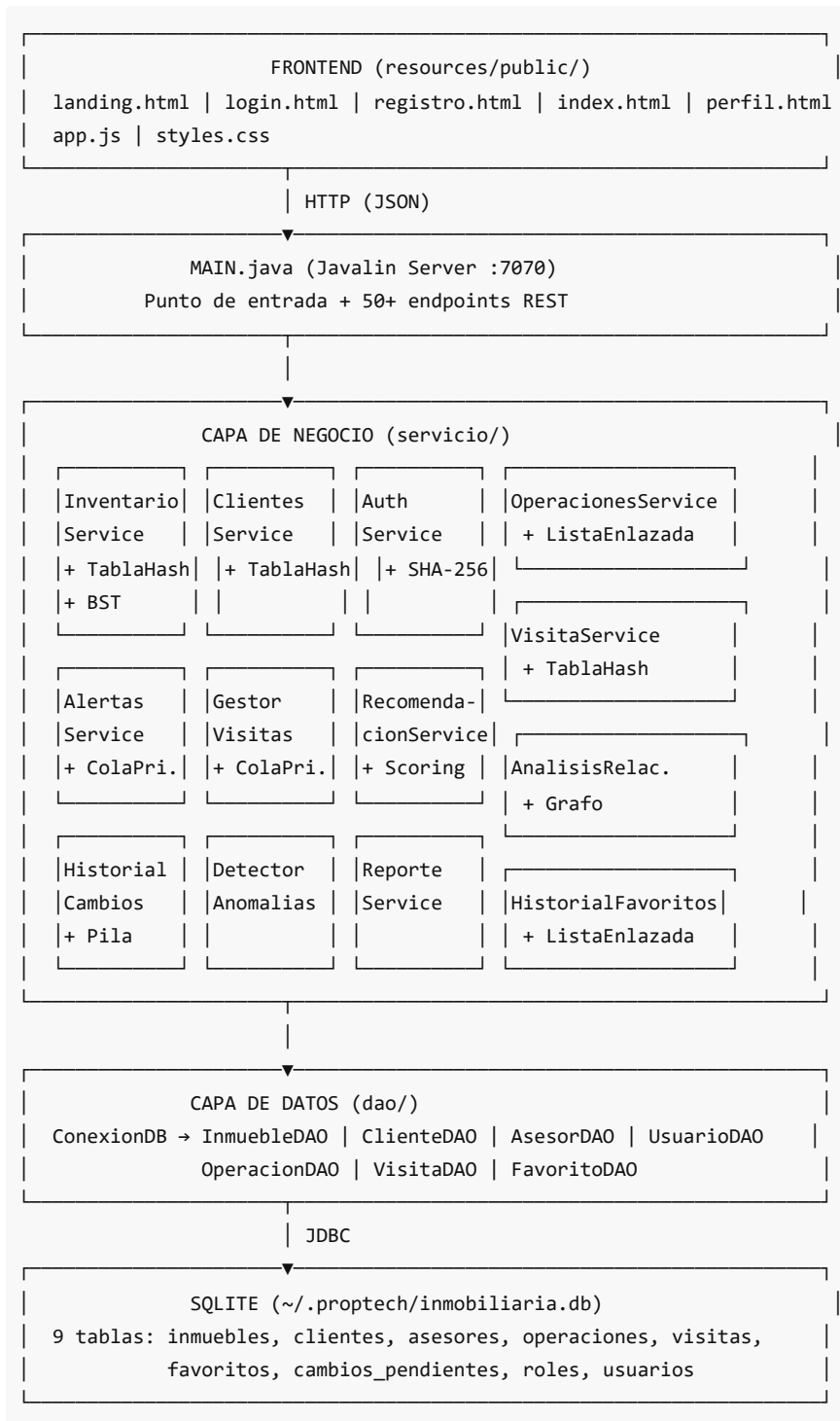
Componente	Tecnología	Versión
Lenguaje	Java	21
Web Framework	Javalin	6.1.3
Servidor Web	Jetty	11.0.18
Base de Datos	SQLite (JDBC)	3.45.2.0
JSON	Jackson Databind	2.16.1
Build	Maven	--
Frontend	Vanilla JavaScript + Bootstrap	5.3
Tests	JUnit Jupiter	5.10.2
Logging	SLF4J Simple	2.0.12

2. Estructura de Directorios

```
ProyectoPropTech/
├── .github/modernize/           # Hooks de desarrollo
├── .vscode/settings.json       # Configuración VS Code
├── test_api.java               # Utilidad CLI para testear API
├──
├── utilidades/                 # Módulo principal (Maven)
│   ├── pom.xml                # Configuración Maven
│   ├── start_server.bat       # Script inicio Windows
│   ├── run_server.bat        # Script ejecución Windows
│   ├── java                   # Script auxiliar
│   ├──
│   ├── src/
│   │   ├── main/java/com/proptech/
│   │   │   ├── Main.java      ★ Punto de entrada (servidor + endpoints)
│   │   │   ├── modelo/        ★ Capa Modelo (8 clases)
│   │   │   ├── dao/           ★ Capa Datos (8 clases)
│   │   │   ├── servicio/      ★ Capa Negocio (13 clases)
│   │   │   └── utilidades/
│   │   │       ├── HashUtils.java
│   │   │       └── estructuras/ ★ 12 estructuras personalizadas
│   │   ├──
│   │   ├── main/resources/public/ ★ Frontend (7 archivos)
│   │   └── test/java/com/proptech/ ★ Tests (4 clases)
```

	target/	# Build output
	docs/	# Documentación

3. Arquitectura en Capas



Patrón de escritura dual: Todos los servicios escriben tanto en SQLite (persistencia) como en estructuras en memoria (rendimiento). Al iniciar el servidor, se cargan los datos desde SQLite hacia las estructuras RAM.

4. Capa Modelo (modelo/)

4.1 Inmueble.java

Atributo	Tipo	Descripción
codigo	String	Identificador único del inmueble
tipo	String	Casa, Apartamento, Local, Oficina, Terreno
direccion	String	Dirección completa
precio	double	Precio de venta o arriendo
area	double	Área en metros cuadrados
estado	String	Disponible, Vendido, Arrendado, Reservado
habitaciones	int	Número de habitaciones
banos	int	Número de baños
tieneParqueadero	boolean	Si cuenta con parqueadero
descripcion	String	Descripción detallada

4.2 Cliente.java

Atributo	Tipo	Descripción
identificacion	String	Número de identificación único
nombre	String	Nombre completo
telefono	String	Teléfono de contacto
presupuestoMaximo	double	Presupuesto máximo disponible
email	String	Correo electrónico
historialConsultas	ListaEnlazada	Historial de propiedades consultadas
favoritos	ListaEnlazada	Lista de propiedades favoritas
tipoInmuebleDeseado	String	Tipo de inmueble que busca
zonasInteres	String	Zonas de interés
cantMinHabitaciones	int	Cantidad mínima de habitaciones

4.3 Asesor.java

Atributo	Tipo	Descripción
----------	------	-------------

idAsesor	String	Identificador único del asesor
nombre	String	Nombre completo
especialidad	String	Especialidad (Ventas, Arriendos, Ambos)
negociosCerrados	int	Cantidad de negocios cerrados
email	String	Correo electrónico
telefono	String	Teléfono de contacto
calificacion	double	Calificación promedio

4.4 Visita.java

Atributo	Tipo	Descripción
idVisita	String	Identificador único de la visita
cliente	Cliente	Cliente que solicita la visita
inmueble	Inmueble	Inmueble a visitar
asesor	Asesor	Asesor asignado
fechaHora	String	Fecha y hora programada
estado	String	Pendiente → Confirmada → Realizada / Cancelada

4.5 Operacion.java

Atributo	Tipo	Descripción
idOperacion	String	Identificador único de la operación
tipo	String	Venta o Arriendo
inmueble	Inmueble	Inmueble involucrado
cliente	Cliente	Cliente involucrado
asesor	Asesor	Asesor que gestionó
monto	double	Monto de la operación
fecha	String	Fecha de la operación

4.6 Alerta.java

Atributo	Tipo	Descripción
idAlerta	String	Identificador único
mensaje	String	Descripción de la alerta
prioridad	int	Nivel de prioridad (menor = más urgente)

fechaCreacion	String	Fecha de generación
---------------	--------	---------------------

4.7 Usuario.java

Atributo	Tipo	Descripción
idUsuario	int	Identificador único autoincremental
email	String	Correo electrónico (único)
passwordHash	String	Hash SHA-256 de la contraseña
rol	Rol	Rol del usuario
nombre	String	Nombre completo
telefono	String	Teléfono
direccion	String	Dirección
intereses	String	Intereses inmobiliarios
fotoPerfil	String	Foto en base64
activo	boolean	Si la cuenta está activa

4.8 Rol.java

Constantes: CLIENTE , VENDEDOR , ADMIN , GERENTE

Atributo	Tipo	Descripción
idRol	int	Identificador único
nombre	String	Nombre del rol

5. Capa de Datos (dao/)

5.1 ConexionDB.java

- **conectar()**: Establece conexión JDBC con SQLite
- **inicializarTablas()**: Crea las 9 tablas si no existen
- Archivo DB: ~/.proptech/inmobiliaria.db

5.2 Esquema de Base de Datos (9 tablas)

```
-- Tabla de inmuebles
CREATE TABLE IF NOT EXISTS inmuebles (
  codigo TEXT PRIMARY KEY,
  tipo TEXT,
  direccion TEXT,
  precio REAL,
  area REAL,
  estado TEXT,
```

```

    habitaciones INTEGER,
    banos INTEGER,
    tieneParqueadero INTEGER,
    descripcion TEXT
);

-- Tabla de clientes
CREATE TABLE IF NOT EXISTS clientes (
    identificacion TEXT PRIMARY KEY,
    nombre TEXT,
    telefono TEXT,
    presupuestoMaximo REAL,
    email TEXT
);

-- Tabla de asesores
CREATE TABLE IF NOT EXISTS asesores (
    idAsesor TEXT PRIMARY KEY,
    nombre TEXT,
    especialidad TEXT,
    email TEXT,
    telefono TEXT,
    calificacion REAL,
    negociosCerrados INTEGER
);

-- Tabla de operaciones
CREATE TABLE IF NOT EXISTS operaciones (
    idOperacion TEXT PRIMARY KEY,
    tipo TEXT,
    idInmueble TEXT,
    idCliente TEXT,
    idAsesor TEXT,
    monto REAL,
    fecha TEXT,
    FOREIGN KEY (idInmueble) REFERENCES inmuebles(codigo),
    FOREIGN KEY (idCliente) REFERENCES clientes(identificacion),
    FOREIGN KEY (idAsesor) REFERENCES asesores(idAsesor)
);

-- Tabla de visitas
CREATE TABLE IF NOT EXISTS visitas (
    idVisita TEXT PRIMARY KEY,
    identificacionCliente TEXT,
    codigoInmueble TEXT,
    idAsesor TEXT,
    fechaHora TEXT,
    estado TEXT,
    FOREIGN KEY (identificacionCliente) REFERENCES clientes(identificacion),
    FOREIGN KEY (codigoInmueble) REFERENCES inmuebles(codigo),
    FOREIGN KEY (idAsesor) REFERENCES asesores(idAsesor)
);

```

```

-- Tabla de favoritos
CREATE TABLE IF NOT EXISTS favoritos (
    identificacionCliente TEXT,
    codigoInmueble TEXT,
    PRIMARY KEY (identificacionCliente, codigoInmueble),
    FOREIGN KEY (identificacionCliente) REFERENCES clientes(identificacion),
    FOREIGN KEY (codigoInmueble) REFERENCES inmuebles(codigo)
);

-- Tabla de cambios pendientes de aprobación
CREATE TABLE IF NOT EXISTS cambios_pendientes (
    id_cambio INTEGER PRIMARY KEY AUTOINCREMENT,
    email_usuario TEXT,
    campo TEXT,
    valor_anterior TEXT,
    valor_nuevo TEXT,
    estado TEXT DEFAULT 'pendiente',
    fecha_solicitud TEXT,
    fecha_revision TEXT,
    revisado_por TEXT
);

-- Tabla de roles
CREATE TABLE IF NOT EXISTS roles (
    id_rol INTEGER PRIMARY KEY AUTOINCREMENT,
    nombre TEXT UNIQUE
);

-- Datos iniciales: CLIENTE, VENDEDOR, ADMIN, GERENTE

-- Tabla de usuarios
CREATE TABLE IF NOT EXISTS usuarios (
    id_usuario INTEGER PRIMARY KEY AUTOINCREMENT,
    email TEXT UNIQUE,
    password_hash TEXT,
    rol_id INTEGER,
    nombre TEXT,
    telefono TEXT,
    direccion TEXT,
    intereses TEXT,
    foto_perfil TEXT,
    activo INTEGER DEFAULT 1,
    FOREIGN KEY (rol_id) REFERENCES roles(id_rol)
);

```

5.3 DAOs - Métodos Principales

DAO	Métodos
InmuebleDAO	guardar(Inmueble), obtenerTodos(), obtenerPorCodigo(String), actualizar(Inmueble), eliminar(String)

ClienteDAO	guardar(Cliente), obtenerPorId(String), obtenerPorEmail(String), obtenerTodos(), actualizar(Cliente), eliminar(String)
AsesorDAO	guardar(Asesor), obtenerPorId(String), obtenerTodos(), actualizar(Asesor), eliminar(String)
UsuarioDAO	crear(Usuario), actualizar(Usuario), buscarPorEmail(String), obtenerTodos(), obtenerRolPorNombre(String)
OperacionDAO	guardar(Operacion), obtenerPorId(String), obtenerTodas(), obtenerPorTipo(String), obtenerPorAsesor(String), obtenerOperacionesPorCliente(String), actualizar(Operacion), eliminar(String)
VisitaDAO	guardar(Visita), obtenerPorId(String), obtenerTodas(), obtenerVisitasPorCliente(String), obtenerVisitasPorAsesor(String), actualizar(Visita), eliminar(String)
FavoritoDAO	agregarFavorito(String, String), quitarFavorito(String, String), obtenerFavoritos(String), esFavorito(String, String)

6. Estructuras de Datos Personalizadas

Todas las estructuras son **genéricas** y se implementaron desde cero sin librerías externas de colecciones.

6.1 Nodo.java

```
Nodo<T>
├─ dato: T
└─ siguiente: Nodo<T>
```

6.2 ListaEnlazada.java

```
ListaEnlazada<T>
├─ cabeza: Nodo<T>
├─ add(T dato)
├─ get(int indice): T
├─ remove(int indice): T
├─ size(): int
└─ imprimir(): void
```

Uso: Historial de operaciones, favoritos de clientes, lista de adyacencia del grafo, listas de resultados varios.

6.3 Pila.java

```
Pila<T>
├─ cima: Nodo<T>
├─ push(T dato)
├─ pop(): T
├─ peek(): T
└─ estaVacia(): boolean
```

Uso: `HistorialCambiosService` — deshacer cambios en propiedades (LIFO).

6.4 Cola.java

```
Cola<T>
├─ frente: Nodo<T>
├─ final: Nodo<T>
├─ enqueue(T dato)
├─ dequeue(): T
└─ estaVacia(): boolean
```

Uso: Disponible para procesos FIFO futuros.

6.5 NodoPrioridad.java

```
NodoPrioridad<T>
├─ dato: T
├─ prioridad: int (menor = más prioritario)
└─ siguiente: NodoPrioridad<T>
```

6.6 ColaPrioridad.java

```
ColaPrioridad<T>
├─ frente: NodoPrioridad<T>
├─ enqueue(T dato, int prioridad)
├─ dequeue(): T
└─ estaVacia(): boolean
```

Uso:

- `AlertasService` : Alertas ordenadas por urgencia
- `GestorVisitasService` : Cola de visitas con prioridad (VIPs primero)

6.7 NodoArbol.java

```
NodoArbol<K, V>
├─ key: K
├─ value: V
├─ izquierdo: NodoArbol<K, V>
└─ derecho: NodoArbol<K, V>
```

6.8 ArbolBinarioBusqueda.java

```
ArbolBinarioBusqueda<K, V>
├─ raiz: NodoArbol<K, V>
├─ insertar(K key, V value)
├─ inorden(ListaEnlazada<V> resultado)
└─ buscar(K key): V
```

Uso: `InventarioInmueblesService` — indexa inmuebles por precio para listados ordenados.

6.9 EntradaHash.java

```

EntradaHash<K, V>
├─ key: K
├─ value: V
└─ siguiente: EntradaHash<K, V>

```

6.10 TablaHash.java

```

TablaHash<K, V>
├─ tabla: EntradaHash<K, V>[capacidad]
├─ put(K key, V value)
├─ get(K key): V
├─ remove(K key): V
├─ containsKey(K key): boolean
├─ size(): int
├─ valores(): ListaEnlazada<V>
└─ keys(): ListaEnlazada<K>

```

Manejo de colisiones mediante **encadenamiento separado**. **Uso (búsquedas $O(1)$):**

- InventarioInmueblesService : inmuebles por código
- ClientesService : clientes por identificación
- OperacionesService : operaciones por ID
- VisitaService : visitas por ID

6.11 Vertice.java

```

Vertice<T>
├─ dato: T
└─ adyacentes: ListaEnlazada<T>

```

6.12 Grafo.java

```

Grafo<T>
├─ vertices: ListaEnlazada<Vertice<T>>
├─ addVertex(T dato)
├─ addEdge(T origen, T destino)
├─ getAdyacentes(T dato): ListaEnlazada<T>
└─ printRelations(): void

```

Grafo **no dirigido** implementado con lista de adyacencia. **Uso:** AnalisisRelacionesService — filtrado colaborativo: "clientes que vieron X también vieron Y".

7. Capa de Negocio — Servicios Detallados

7.1 Mapeo Servicio ↔ Estructuras de Datos

Servicio	Estructuras en Memoria	Propósito
InventarioInmueblesService	TablaHash<String, Inmueble> + ArbolBinarioBusqueda<Double, Inmueble>	Catálogo de propiedades

CientesService	TablaHash<String, Cliente>	Registro de clientes
OperacionesService	TablaHash<String, Operacion> + ListaEnlazada	Gestión de transacciones
VisitaService	TablaHash<String, Visita>	Programación de visitas
AlertasService	ColaPrioridad	Alertas automáticas
GestorVisitasService	ColaPrioridad	Cola de visitas prioritarias
HistorialCambiosService	Pila	Deshacer cambios
AnalisisRelacionesService	Grafo	Filtrado colaborativo
HistorialYFavoritosService	ListaEnlazada (embebida en Cliente)	Favoritos e historial

7.2 Detalle de Servicios

InventarioInmueblesService

- `agregarInmueble(Inmueble)` : Agrega a TablaHash (por código) y BST (por precio)
- `buscarPorCodigo(String)` : Búsqueda O(1) en TablaHash
- `obtenerOrdenadosPorPrecio()` : Recorrido inorden del BST → ListaEnlazada ordenada
- `obtenerTodos()` : Todos los inmuebles
- `eliminarInmueble(String)` : Elimina de ambas estructuras

CientesService

- `agregarCliente(Cliente)` : Agrega a TablaHash por identificación
- `buscarPorId(String)` : Búsqueda O(1)
- `obtenerTodos()` : Todos los clientes
- `actualizar(Cliente)` : Actualiza en TablaHash

OperacionesService

- `registrarOperacion(Operacion)` : Inserta en TablaHash + ListaEnlazada. Actualiza estado del inmueble a Vendido/Arrendado. Incrementa negociosCerrados del asesor.
- `buscarPorId(String)` : O(1)
- `obtenerHistorial()` : Recorre ListaEnlazada
- `obtenerPorTipo(String)` : Filtra por Venta/Arriendo
- `cancelarOperacion(String)` : Elimina de estructuras y revierte estado del inmueble

VisitaService

- `agendarVisita(Visita)` : Valida conflicto de horario (±2h del asesor). Inserta en TablaHash. Crea cliente automático si no existe.
- `confirmarVisita(String)` : Pendiente → Confirmada
- `realizarVisita(String)` : Confirmada → Realizada
- `cancelarVisita(String)` : Pendiente/Confirmada → Cancelada
- `obtenerVisitasPorCliente(String)` : Filtradas por email
- `obtenerVisitasPorAsesor(String)` : Filtradas por ID

AlertasService

Genera **6 tipos de alertas** automáticas con prioridad:

1. Contratos próximos a vencer (prioridad 1)
2. Inmuebles sin visitas en 30+ días (prioridad 2)
3. Zonas con alta demanda (prioridad 3)
4. Clientes con visitas pendientes de confirmación (prioridad 4)
5. Inmuebles reservados sin concretar (prioridad 5)
6. Clientes sin seguimiento reciente (prioridad 6)

Las alertas se encolan en `ColaPrioridad<Alerta>` y se desencolan con `siguienteAlerta()`.

GestorVisitasService

- `encolarVisita(Visita, int prioridad)` : Encola con prioridad (VIP = 1, normal = 5)
- `desencolarVisita()` : Desencola la más prioritaria
- `obtenerSiguiente()` : Consulta sin desencolar

RecomendacionService

- `recomendar(Inmuelle[], Cliente)` : Evalúa 6 criterios de puntuación:
 1. Presupuesto: $\text{puntuaje} = 1 - |\text{precio} - \text{presupuesto}| / \text{presupuesto}$
 2. Tipo de inmueble: +30 si coincide
 3. Zona de interés: +20 si coincide
 4. Habitaciones mínimas: +15 si cumple
 5. Historial de consultas: +10 por tipo consultado antes
 6. Favoritos: +5 por tipo en favoritos
- Retorna top 10 en `ListaEnlazada<InmuelleConPuntuacion>`

AnalisisRelacionesService

- `registrarInteres(String clienteId, String inmuebleCodigo)` : Construye grafo donde clientes e inmuebles son vértices, y las visitas/consultas son aristas
- `recomendarRelacionados(String inmuebleCodigo)` : Encuentra otros inmuebles vistos por clientes que vieron este inmueble
- Basado en **filtrado colaborativo** (grafo bipartito)

HistorialCambiosService

- `guardarCambio(String codigoInmuelle, String campo, String valorAnterior, String valorNuevo)` : Push a la Pila
- `deshacerCambio()` : Pop de la Pila y reversión
- `obtenerHistorialCambios()` : Lista todo sin desapilar

HistorialYFavoritosService

- `agregarConsulta(Cliente, Inmuelle)` : Agrega al historial del cliente
- `toggleFavorito(Cliente, Inmuelle)` : Agrega o quita de favoritos
- `obtenerFavoritos(Cliente)` : Lista favoritos

DetectorAnomaliesService

Detecta 4 tipos de anomalías:

1. **Visitas sin cierre**: Clientes con múltiples visitas pero ninguna operación cerrada
2. **Sobrecarga de asesores**: Asesores con más de 5 visitas asignadas en un período
3. **Cambios de precio frecuentes**: Inmuebles con más de 3 cambios de precio

4. **Concentración geográfica:** Inmuebles con alta frecuencia de consultas/visitas

ReporteService

5 tipos de reportes:

1. **Rendimiento de inmuebles:** Ventas/arriendos por período
2. **Rendimiento de asesores:** Métricas por asesor
3. **Precios por zona:** Promedio, mínimo, máximo por zona geográfica
4. **Clientes activos:** Clientes con más consultas/visitas
5. **Tipos de operación:** Distribución Venta vs Arriendo

Además, reporte filtrado por: tipo de inmueble, zona, rango de precio.

AuthService

- registrar(Usuario) : Hash SHA-256 de contraseña, asigna rol CLIENTE, persiste
- autenticar(email, password) : Verifica hash, retorna usuario
- crearAdmin() : Admin por defecto (admin@proptech.com / admin123)
- simularEnvioRecomendaciones(email) : Envía 3 recomendaciones post-registro

8. API REST Completa

Todas las rutas están definidas en `Main.java` y escuchan en `http://localhost:7070` .

8.1 INMUEBLES

`GET /api/inmuebles`

Lista todos los inmuebles.

Respuesta exitosa (200):

```
[
  {
    "codigo": "APT-001",
    "tipo": "Apartamento",
    "direccion": "Calle 123 #45-67",
    "precio": 250000000,
    "area": 80.5,
    "estado": "Disponible",
    "habitaciones": 3,
    "banos": 2,
    "tieneParqueadero": true,
    "descripcion": "Hermoso apartamento en zona residencial"
  }
]
```

`GET /api/inmuebles/{codigo}`

Busca un inmueble por su código. Implementa **búsqueda O(1)** mediante `TablaHash<String, Inmueble>` .

Parámetros:

- `codigo` (path): Código del inmueble

Respuesta exitosa (200): Objeto Inmueble JSON **Respuesta error (404):** `{ "error": "Inmueble no encontrado" }`

POST /api/inmuebles

Crea un nuevo inmueble.

Cuerpo de la solicitud:

```
{
  "codigo": "CASA-010",
  "tipo": "Casa",
  "direccion": "Carrera 15 #30-20",
  "precio": 550000000,
  "area": 200,
  "estado": "Disponible",
  "habitaciones": 5,
  "banos": 3,
  "tieneParqueadero": true,
  "descripcion": "Casa amplia con jardín y piscina"
}
```

Reglas de negocio:

- Valida que todos los campos requeridos estén presentes
- Agrega a la `TablaHash` y al `ArbolBinarioBusqueda` (indexado por precio)
- Persiste en SQLite mediante `InmuebleDAO.guardar()`

Respuesta exitosa (201): `{ "mensaje": "Inmueble agregado exitosamente" }`

PUT /api/inmuebles/{codigo}

Actualiza un inmueble existente.

Parámetros:

- `codigo` (path): Código del inmueble a actualizar

Cuerpo de la solicitud: Objeto Inmueble parcial o completo

Reglas de negocio:

- Si el estado cambia a "Vendido" o "Arrendado", **automáticamente crea una operación** de tipo Venta o Arriendo asociada al cliente que tenía el inmueble en seguimiento
- Si hay cambio de precio, se registra en el `HistorialCambiosService` para posible deshacer
- Si el asesor asignado tiene el inmueble como suyo, se incrementa su contador `negociosCerrados`
- Persiste actualización en SQLite

Respuesta exitosa (200): `{ "mensaje": "Inmueble actualizado exitosamente" }`

DELETE /api/inmuebles/{codigo}

Elimina un inmueble.

Parámetros:

- `codigo` (path): Código del inmueble

Reglas de negocio:

- Elimina de `TablaHash` y `ArbolBinarioBusqueda`
- Elimina de SQLite
- Limpia referencias en favoritos de clientes

Respuesta exitosa (200): { "mensaje": "Inmueble eliminado" }

8.2 CLIENTES

GET /api/clientes

Lista todos los clientes.

Respuesta (200): Array de objetos Cliente JSON

GET /api/clientes/{id}

Busca un cliente por identificación. **Búsqueda O(1)** mediante `TablaHash<String, Cliente>` .

Parámetros:

- `id` (path): Número de identificación

Respuesta (200): Objeto Cliente JSON **Respuesta (404):** { "error": "Cliente no encontrado" }

POST /api/clientes

Crea un nuevo cliente.

Cuerpo:

```
{
  "identificacion": "1234567890",
  "nombre": "Juan Pérez",
  "telefono": "3001234567",
  "presupuestoMaximo": 300000000,
  "email": "juan@email.com",
  "tipoInmuebleDeseado": "Apartamento",
  "zonasInteres": "Norte, Centro",
  "cantMinHabitaciones": 3
}
```

Reglas de negocio:

- Agrega a `TablaHash` y SQLite
- Inicializa `historialConsultas` y `favoritos` como `ListaEnlazada` vacía

Respuesta (201): { "mensaje": "Cliente creado exitosamente" }

POST /api/clientes/seed

Re-siembramos datos de prueba (clientes de demostración).

Respuesta (200): { "mensaje": "Datos de prueba insertados" }

PUT /api/clientes/{id}

Actualiza un cliente.

Reglas de negocio:

- Actualiza en TablaHash y SQLite

Respuesta (200): { "mensaje": "Cliente actualizado" }

DELETE /api/clientes/{id}

Elimina un cliente.

Reglas de negocio:

- Elimina de TablaHash y SQLite
- Limpia referencias en favoritos, visitas y operaciones

Respuesta (200): { "mensaje": "Cliente eliminado" }

GET /api/clientes/perfil?email={email}

Obtiene perfil de cliente por email.

Parámetros query:

- email : Correo electrónico del cliente

Respuesta (200): Objeto Cliente JSON (con historialConsultas y favoritos poblados)

PUT /api/clientes/perfil?email={email}

Actualiza perfil de cliente.

Cuerpo:

```
{
  "nombre": "Juan Carlos Pérez",
  "telefono": "3009876543",
  "presupuestoMaximo": 350000000,
  "tipoInmuebleDeseado": "Casa",
  "zonasInteres": "Sur",
  "cantMinHabitaciones": 4
}
```

Respuesta (200): { "mensaje": "Perfil actualizado" }

8.3 VISITAS

POST /api/visitas

Agenda una nueva visita.

Cuerpo:

```
{
  "inmuebleCodigo": "APT-001",
  "idAsesor": "ASE-001",
  "fechaHora": "2025-06-15 10:00",
  "cliente": {
    "identificacion": "1234567890",
    "nombre": "María López",
    "telefono": "3105551234",
    "email": "maria@email.com"
  }
}
```

Reglas de negocio:

- **Detección de conflictos horarios:** Si el asesor ya tiene una visita programada en un rango de ± 2 horas, la solicitud es rechazada
 - Lógica implementada en el helper `fechasConConflicto()` en `Main.java`
 - Compara fechas como strings en formato "yyyy-MM-dd HH:mm"
 - Si hay diferencia < 2 horas $\rightarrow$ conflicto
- Si el cliente no existe en el sistema, **se crea automáticamente** con los datos proporcionados
- Asigna estado "Pendiente" a la visita
- Almacena en `VisitaService.TablaHash<String, Visita>` y en SQLite
- Registra el interés en `AnalisisRelacionesService` (construye el grafo)

Respuesta exitosa (201):

```
{
  "mensaje": "Visita agendada exitosamente",
  "idVisita": "VIS-abc123"
}
```

Respuesta conflicto (409):

```
{
  "error": "El asesor ya tiene una visita programada cerca de esa fecha y hora"
}
```

GET /api/visitas/cliente/{email}

Obtiene todas las visitas de un cliente.

Parámetros:

- `email` (path): Email del cliente

Respuesta (200): Array de objetos Visita con datos del inmueble y asesor (JOIN query)

GET /api/visitas/asesor/{idAsesor}

Obtiene todas las visitas asignadas a un asesor.

Parámetros:

- `idAsesor` (path): ID del asesor

Respuesta (200): Array de objetos Visita

PUT /api/visitas/{idVisita}/confirmar

Confirma una visita (Pendiente → Confirmada).

Parámetros:

- `idVisita` (path): ID de la visita

Respuesta (200): { "mensaje": "Visita confirmada" }

PUT /api/visitas/{idVisita}/realizar

Marca visita como realizada (Confirmada → Realizada).

Parámetros:

- `idVisita` (path): ID de la visita

Respuesta (200): { "mensaje": "Visita realizada" }

PUT /api/visitas/{idVisita}/cancelar

Cancela una visita (Pendiente o Confirmada → Cancelada).

Parámetros:

- `idVisita` (path): ID de la visita

Respuesta (200): { "mensaje": "Visita cancelada" }

8.4 OPERACIONES

GET /api/operaciones

Lista todas las operaciones.

Respuesta (200): Array de objetos Operacion JSON

GET /api/operaciones/{id}

Busca operación por ID. **Búsqueda O(1)** mediante `TablaHash<String, Operacion>`.

Parámetros:

- `id` (path): ID de la operación

Respuesta (200): Objeto Operacion JSON **Respuesta (404):** { "error": "Operación no encontrada" }

POST /api/operaciones

Registra una nueva operación (Venta o Arriendo).

Cuerpo:

```
{
  "tipo": "Venta",
  "inmueble": { "codigo": "APT-001" },
  "cliente": { "identificacion": "1234567890" },
  "asesor": { "idAsesor": "ASE-001" },
  "monto": 245000000,
  "fecha": "2025-06-01"
}
```

Reglas de negocio:

- Cambia el estado del inmueble a "Vendido" o "Arrendado"
- Incrementa `negociosCerrados` del asesor en 1
- Almacena en `TablaHash<String, Operacion>` y `ListaEnlazada<Operacion>` (historial cronológico)
- Persiste en SQLite

Respuesta (201):

```
{
  "mensaje": "Operación registrada exitosamente",
  "idOperacion": "OP-abc123"
}
```

POST /api/operaciones/renovacion

Registra renovación de un contrato de arriendo.

Cuerpo:

```
{
  "idOperacionOriginal": "OP-abc123",
  "nuevoMonto": 260000000,
  "nuevaFecha": "2026-06-01"
}
```

Respuesta (201): { "mensaje": "Renovación registrada", "idOperacion": "OP-..." }

POST /api/operaciones/cancelacion

Cancela una operación existente.

Cuerpo:

```
{
  "idOperacion": "OP-abc123"
}
```

```
}
```

Reglas de negocio:

- Revierte el estado del inmueble a "Disponible"
- Elimina de estructuras en memoria y SQLite

Respuesta (200): { "mensaje": "Operación cancelada" }

GET /api/operaciones/tipo/{tipo}

Filtra operaciones por tipo.

Parámetros:

- `tipo` (path): "Venta" o "Arriendo"

Respuesta (200): Array filtrado de operaciones

GET /api/operaciones/asesor/{idAsesor}

Filtra operaciones por asesor.

Parámetros:

- `idAsesor` (path): ID del asesor

Respuesta (200): Array filtrado de operaciones

8.5 AUTENTICACIÓN

POST /api/auth/registro

Registra un nuevo usuario en el sistema.

Cuerpo:

```
{
  "email": "nuevo@email.com",
  "password": "miPassword123",
  "nombre": "Carlos López",
  "telefono": "3001112233"
}
```

Reglas de negocio:

- Aplica **SHA-256** a la contraseña mediante `HashUtils.hashPassword()`
- Asigna automáticamente rol **CLIENTE**
- Crea en las tablas `usuarios` y `clientes`
- **Simula envío de correo** con 3 recomendaciones de inmuebles (basadas en `RecomendacionService`)
- Persiste en SQLite mediante `UsuarioDAO.crear()`

Respuesta (201):

```
{
  "mensaje": "Usuario registrado exitosamente",
  "recomendaciones": [ 3 inmuebles recomendados ]
}
```

POST /api/auth/login

Inicia sesión de usuario.

Cuerpo:

```
{
  "email": "admin@proptech.com",
  "password": "admin123"
}
```

Reglas de negocio:

- Verifica el hash SHA-256 contra la base de datos
- Retorna datos completos del usuario (incluyendo rol, nombre, foto, etc.)

Respuesta exitosa (200):

```
{
  "idUsuario": 1,
  "email": "admin@proptech.com",
  "nombre": "Admin PropTech",
  "rol": "ADMIN",
  "activo": true
}
```

Respuesta error (401):

```
{
  "error": "Credenciales inválidas"
}
```

8.6 USUARIO / PERFIL

GET /api/usuario/perfil?email={email}

Obtiene el perfil completo de un usuario.

Parámetros query:

- email : Correo del usuario

Respuesta (200): Objeto Usuario JSON (incluye fotoPerfil en base64)

PUT /api/usuario/actualizar?email={email}

Actualiza perfil de usuario.

Cuerpo:

```
{
  "nombre": "Nuevo Nombre",
  "telefono": "3115559999",
  "direccion": "Calle Nueva #1-23",
  "intereses": "Casas, Apartamentos",
  "fotoPerfil": "data:image/png;base64,..."
}
```

Reglas de negocio:

- Los cambios en `intereses` y `fotoPerfil` se aplican **inmediatamente**
- Los cambios en `nombre`, `telefono`, `direccion` requieren **aprobación del administrador**
 - Se crea un registro en `cambios_pendientes` con estado "pendiente"
 - El admin debe aprobar o rechazar mediante los endpoints de admin
- Si hay cambios que requieren aprobación, se notifica al usuario

Respuesta (200):

```
{
  "mensaje": "Perfil actualizado. Algunos cambios requieren aprobación del administrador.",
  "cambiosPendientes": ["nombre", "telefono", "direccion"]
}
```

GET /api/usuarios

Lista todos los usuarios registrados.

Respuesta (200): Array de objetos Usuario JSON

8.7 FAVORITOS

POST /api/favoritos/{codigoInmueble}?email={email}

Toggle de favorito: si el inmueble está en favoritos lo quita, si no está lo agrega.

Parámetros:

- `codigoInmueble` (path): Código del inmueble
- `email` (query): Email del usuario

Reglas de negocio:

- Agrega/quita en la `ListaEnlazada<Inmueble>` favoritos del cliente
- Persiste en SQLite mediante `FavoritoDAO`

Respuesta exitosa (200):

```
{
  "mensaje": "Favorito agregado",
}
```

```
"esFavorito": true
}
```

GET /api/favoritos?email={email}

Lista los favoritos del usuario.

Parámetros query:

- email : Email del usuario

Respuesta (200): Array de objetos Inmueble JSON

8.8 RECOMENDACIONES

GET /api/recomendaciones/{idCliente}

Obtiene las 10 mejores recomendaciones para un cliente.

Parámetros:

- idCliente (path): Identificación del cliente

Algoritmo de recomendación (6 criterios):

1. **Compatibilidad de presupuesto** (peso: normalizado a 0-1): $\text{puntaje} = 1 - |\text{precioInmueble} - \text{presupuestoCliente}| / \text{presupuestoCliente}$
2. **Coincidencia de tipo** (peso: +30): Si `inmueble.tipo == cliente.tipoInmuebleDeseado` → +30 puntos
3. **Coincidencia de zona** (peso: +20): Si la dirección del inmueble contiene alguna zona de `cliente.zonasInteres` → +20 puntos
4. **Habitaciones suficientes** (peso: +15): Si `inmueble.habitaciones >= cliente.cantMinHabitaciones` → +15 puntos
5. **Historial de consultas** (peso: +10): Si el cliente ha consultado un inmueble del mismo tipo antes → +10 puntos
6. **Favoritos** (peso: +5): Si el cliente tiene un favorito del mismo tipo → +5 puntos

Cálculo final: Suma ponderada de todos los criterios. **Resultado:** Top 10 inmuebles ordenados por puntuación descendente.

Respuesta (200):

```
[
  { "inmueble": { ... }, "puntuacion": 85.5 },
  { "inmueble": { ... }, "puntuacion": 72.3 }
]
```

8.9 ANOMALÍAS

GET /api/anomalias/visitas-sin-cierre

Detecta clientes con múltiples visitas pero **ninguna operación cerrada**.

Lógica: Recorre todas las visitas agrupadas por cliente y verifica si existe al menos una operación para ese cliente.

Respuesta (200):

```
{
  "tipo": "Visitas sin cierre",
  "total": 3,
  "datos": [
    { "cliente": "1234567890", "nombre": "Juan Pérez", "totalVisitas": 5,
      "operacionesCerradas": 0 }
  ]
}
```

GET /api/anomalias/sobrecarga-asesores

Detecta asesores con más de 5 visitas asignadas.

Lógica: Recorre las visitas agrupadas por asesor, cuenta las pendientes/confirmadas.

Respuesta (200):

```
{
  "tipo": "Sobrecarga de asesores",
  "total": 1,
  "datos": [
    { "asesor": "ASE-001", "nombre": "Carlos Pérez", "visitasAsignadas": 8 }
  ]
}
```

GET /api/anomalias/cambios-precio

Detecta inmuebles con más de 3 cambios de precio registrados.

Lógica: Consulta `HistorialCambiosService` (Pila de cambios) para contar cambios de precio por inmueble.

Respuesta (200):

```
{
  "tipo": "Cambios de precio frecuentes",
  "total": 2,
  "datos": [
    { "inmueble": "APT-001", "direccion": "Calle 123", "cambiosPrecio": 5 }
  ]
}
```

GET /api/anomalias/concentracion-geografica

Detecta inmuebles con alta frecuencia de visitas/consultas (posible burbuja o irregularidad).

Lógica: Cuenta visitas por inmueble, aquellos por encima de un umbral.

Respuesta (200):


```
{
  "tipo": "Concentración geográfica",
  "total": 2,
  "datos": [
    { "inmueble": "APT-001", "direccion": "Calle 123", "totalVisitas": 12 }
  ]
}
```

GET /api/anomalias/total

Retorna el **consolidado de todas las anomalías** detectadas.

Respuesta (200):

```
{
  "visitasSinCierre": { "total": 3, "datos": [...] },
  "sobrecargaAsesores": { "total": 1, "datos": [...] },
  "cambiosPrecio": { "total": 2, "datos": [...] },
  "concentracionGeografica": { "total": 2, "datos": [...] }
}
```

8.10 REPORTES

GET /api/reportes/rendimiento-inmuebles

Reporte de rendimiento de inmuebles: cantidad de ventas, arriendos y total por período.

Respuesta (200):

```
{
  "totalVentas": 5,
  "totalArriendos": 3,
  "montoTotalVentas": 125000000,
  "montoTotalArriendos": 45000000,
  "inmuebles": [ ... lista detallada ... ]
}
```

GET /api/reportes/asesores

Reporte de rendimiento de asesores: métricas individuales.

Respuesta (200):

```
[
  {
    "idAsesor": "ASE-001",
    "nombre": "Carlos Pérez",
    "negociosCerrados": 8,
    "calificacion": 4.5,
    "montoTotalGestionado": 250000000,
  }
]
```

```
    "operaciones": [ ... ]
  }
]
```

GET /api/reportes/precios-zona

Reporte de precios promedio, mínimo y máximo por zona.

Respuesta (200):

```
[
  {
    "zona": "Norte",
    "precioPromedio": 350000000,
    "precioMinimo": 150000000,
    "precioMaximo": 800000000,
    "cantidadInmuebles": 12
  }
]
```

GET /api/reportes/clientes-activos

Clientes con mayor actividad (consultas, visitas, operaciones).

Respuesta (200):

```
[
  {
    "cliente": "1234567890",
    "nombre": "Juan Pérez",
    "totalConsultas": 15,
    "totalVisitas": 5,
    "totalOperaciones": 1
  }
]
```

GET /api/reportes/tipos-operacion

Distribución de tipos de operación (Venta vs Arriendo).

Respuesta (200):

```
{
  "ventas": {
    "cantidad": 5,
    "montoTotal": 1250000000,
    "porcentaje": 62.5
  },
  "arriendos": {
    "cantidad": 3,
    "montoTotal": 45000000,
  }
}
```

```
    "porcentaje": 37.5
  }
}
```

GET /api/reportes/rendimiento-filtrado?tipo={tipo}&zona={zona}&precioMin={min}&precioMax={max}

Reporte de rendimiento con filtros avanzados.

Parámetros query (opcionales):

- tipo : Tipo de inmueble (Casa, Apartamento, etc.)
- zona : Zona geográfica
- precioMin : Precio mínimo
- precioMax : Precio máximo

Respuesta (200): Array filtrado de operaciones

8.11 ADMINISTRACIÓN

GET /api/admin/cambios-pendientes

Lista todos los cambios de perfil pendientes de aprobación.

Respuesta (200):

```
[
  {
    "id_cambio": 1,
    "email_usuario": "usuario@email.com",
    "campo": "nombre",
    "valor_anterior": "Carlos",
    "valor_nuevo": "Carlos Andrés",
    "estado": "pendiente",
    "fecha_solicitud": "2025-06-10 14:30:00"
  }
]
```

PUT /api/admin/cambios-pendientes/{id}/aprobar?adminEmail={email}

Aprueba un cambio pendiente.

Parámetros:

- id (path): ID del cambio pendiente
- adminEmail (query): Email del administrador que aprueba

Reglas de negocio:

- Actualiza el campo correspondiente en la tabla usuarios
- Marca el cambio como "aprobado" con fecha y admin responsable

Respuesta (200): { "mensaje": "Cambio aprobado exitosamente" }

PUT /api/admin/cambios-pendientes/{id}/rechazar?adminEmail={email}

Rechaza un cambio pendiente.

Parámetros:

- `id` (path): ID del cambio pendiente
- `adminEmail` (query): Email del administrador

Reglas de negocio:

- NO modifica el valor en `usuarios`
- Marca el cambio como "rechazado"

Respuesta (200): `{ "mensaje": "Cambio rechazado" }`

8.12 ALERTAS

`GET /api/alertas/siguiente`

Desencola y retorna la alerta más urgente de la `ColaPrioridad<Alerta>` .

Reglas de negocio:

- Las alertas se generan automáticamente al iniciar el servidor o bajo demanda
- Prioridad 1 = más urgente (contratos próximos a vencer)
- Prioridad 6 = menos urgente (clientes sin seguimiento)

Respuesta (200):

```
{
  "idAlerta": "ALT-001",
  "mensaje": "Contrato de arriendo APT-001 vence en 5 días",
  "prioridad": 1,
  "fechaCreacion": "2025-06-10"
}
```

Respuesta vacía (200) si no hay alertas:

```
{
  "mensaje": "No hay alertas pendientes"
}
```

8.13 ASESORES Y DASHBOARD

`GET /api/asesores`

Lista todos los asesores.

Respuesta (200): Array de objetos Asesor JSON

`GET /api/asesor/dashboard?idAsesor={idAsesor}`

Obtiene métricas del dashboard de un asesor.

Parámetros query:

- `idAsesor` : ID del asesor

Respuesta (200):

```
{
  "asesor": { ... datos del asesor ... },
  "totalVisitasAsignadas": 8,
  "visitasPendientes": 3,
  "visitasRealizadas": 4,
  "visitasCanceladas": 1,
  "totalOperacionesCerradas": 8,
  "montoTotalGestionado": 2500000000,
  "inmueblesAsignados": [ ... ]
}
```

8.14 DIAGNÓSTICO

`GET /api/diagnostico/usuarios`

Endpoint de diagnóstico que lista usuarios directamente desde la BD.

Respuesta (200): Array de usuarios en crudo

8.15 RUTA RAÍZ

`GET /`

Redirige al archivo estático `landing.html` (página de aterrizaje).

9. Roles y Permisos

Recurso	CLIENTE	VENDEDOR	ADMIN	GERENTE
Ver inmuebles	✓	✓	✓	✓
Agregar/Editar inmuebles	✗	✓	✓	✗
Eliminar inmuebles	✗	✗	✓	✗
Agendar visitas	✓	✓	✓	✓
Confirmar/Cancelar visitas	✓ (propias)	✓ (asignadas)	✓	✓
Ver dashboard asesor	✗	✓	✓	✗
Ver reportes	✗	✗	✓	✓
Ver anomalías	✗	✗	✓	✓
Aprobar cambios perfil	✗	✗	✓	✗
Favoritos	✓	✗	✗	✗
Recomendaciones	✓	✗	✗	✗

Nota: El control de roles está implementado principalmente en el frontend (visibilidad de botones/secciones según `user.rol`). El backend tiene validaciones limitadas (ej. solo admin puede editar inmuebles vendidos).

10. Frontend

Archivos (en `src/main/resources/public/`)

Archivo	Descripción
landing.html	Página de aterrizaje con hero, características, testimonios y formulario de registro. Marca: "Insignia Inmo"
login.html	Formulario de inicio de sesión → almacena usuario en localStorage → redirige a index.html
registro.html	Formulario de registro con confirmación de contraseña → POST a <code>/api/auth/registro</code> → muestra 3 recomendaciones
index.html	SPA principal con navbar por roles, 4 pestañas (Inmuebles, Clientes, Operaciones, Reportes) + admin approvals + 6 modales
perfil.html	Edición de perfil con foto (base64), nombre, email, teléfono, dirección, intereses. Logout.
app.js (1573 líneas)	Lógica completa: <code>initApp()</code> con UI por roles, navegación, CRUD, visitas, favoritos, dashboard, reportes, admin, notificaciones toast
styles.css (244 líneas)	Sistema de diseño con variables CSS, paleta (<code>#B59E81</code> , <code>#4A4A48</code>), animaciones, responsive

Gestión de Sesión

- El frontend almacena el usuario en `localStorage` como `user` (objeto JSON con email, rol, nombre, etc.)
- El backend es **stateless**: cada request lleva el email del usuario como query parameter
- La UI se adapta según `user.rol` (oculta/muestra botones y secciones)

11. Tests

Archivo	Pruebas	Descripción
EstructurasTest.java	3 tests	<code>probarListaEnlazada()</code> , <code>probarPilaDeshacer()</code> , <code>probarTablaHash()</code>
DatabaseTest.java	2 tests	<code>probarGuardadoYLecturaEnSQL()</code> , <code>verificarUsuariosPersisten()</code>
FullTest.java	1 test	<code>testFullConexionYDatos()</code> — inicialización + carga end-to-end
ServiciosTest.java	4 tests	<code>probarInventarioInmuebles()</code> , <code>probarGestorVisitasPrioridad()</code> , <code>probarHistorialDeshacerCambios()</code> , <code>probarMotorRecomendaciones()</code>

Total: 10 pruebas unitarias con JUnit Jupiter 5.10.2.

12. Flujo de Datos - Ejemplos Complejos

12.1 Flujo: Compra de Inmueble

```
Cliente busca inmueble → GET /api/inmuebles
      ↓
Cliente agenda visita → POST /api/visitas (conflicto horario? no)
      ↓
Asesor confirma visita → PUT /api/visitas/{id}/confirmar
      ↓
Visita realizada → PUT /api/visitas/{id}/realizar
      ↓
Cliente decide comprar → POST /api/operaciones (tipo: Venta)
      ↓
Sistema automáticamente:
- Cambia estado inmueble → "Vendido"
- Incrementa negociosCerrados del asesor
- Almacena en TablaHash + ListaEnlazada + SQLite
```

12.2 Flujo: Detección de Anomalías

```
Admin solicita anomalías → GET /api/anomalias/total
      ↓
DetectorAnomaliesService ejecuta 4 análisis:
  1. Visitas sin cierre → Recorre visitas vs operaciones por cliente
  2. Sobrecarga asesores → Cuenta visitas pendientes por asesor
  3. Cambios precio → Consulta Pila de HistorialCambiosService
  4. Concentración geográfica → Cuenta visitas por inmueble
      ↓
Retorna JSON consolidado → Frontend muestra tarjetas de anomalías
```

12.3 Flujo: Recomendación Personalizada

```
Cliente entra al sistema → GET /api/recomendaciones/{id}
      ↓
RecomendacionService obtiene:
- Todos los inmuebles disponibles (desde TablaHash)
- Datos del cliente (desde TablaHash)
      ↓
Para cada inmueble, calcula puntuación (6 criterios):
|-> Presupuesto (0-1 normalizado)
|-> Tipo (+30 si coincide)
|-> Zona (+20 si coincide)
|-> Habitaciones (+15 si suficiente)
|-> Historial (+10 si mismo tipo consultado)
|-> Favoritos (+5 si mismo tipo en favoritos)
      ↓
Ordena por puntuación descendente → Retorna top 10
```

13. Configuración del Proyecto

pom.xml

```
<groupId>com.proptech</groupId>
<artifactId>inmobiliaria</artifactId>
<version>1.0-SNAPSHOT</version>
<mainClass>com.proptech.Main</mainClass>
```

Compilación y Ejecución

```
cd utilidades
mvn clean package          # Compila y empaqueta JAR
mvn exec:java              # Ejecuta directamente
java -jar target/inmobiliaria-1.0-SNAPSHOT.jar # Ejecuta JAR
```

Scripts Windows

- start_server.bat : Inicia el servidor
- run_server.bat : Ejecuta la aplicación

Archivo BD

```
~/proptech/inmobiliaria.db
```

14. Resumen de Estructuras de Datos vs Funcionalidad

Necesidad de Negocio	Estructura	Operación	Complejidad
Buscar inmueble por código	TablaHash<String, Inmueble>	get(key)	O(1)
Buscar cliente por ID	TablaHash<String, Cliente>	get(key)	O(1)
Buscar operación por ID	TablaHash<String, Operacion>	get(key)	O(1)
Buscar visita por ID	TablaHash<String, Visita>	get(key)	O(1)
Listar inmuebles ordenados por precio	ArbolBinarioBusqueda<Double, Inmueble>	inorden()	O(n)
Alertas urgentes primero	ColaPrioridad	dequeue()	O(1)
Cola de visitas VIP	ColaPrioridad	dequeue()	O(1)
Deshacer cambios de precio	Pila	pop()	O(1)
Historial cronológico de operaciones	ListaEnlazada	add() / recorrido	O(1) / O(n)
Favoritos del cliente	ListaEnlazada	add() / remove()	O(1) / O(n)
"Quien vio X también vio Y"	Grafo	adyacentes()	O(1)

